arXiv:2601.17354v5 [cs.CV] 27 May 2026

PocketGS: High-Fidelity On-Device Training for 3D Gaussian Splatting

Wenzhi Guo, Guangchi Fang, Shu Yang, Bing Wang*

Fig. 1: Our PocketGS enables high-quality end-to-end 3DGS reconstruction on commodity smartphones. Compared to standard 3DGS workstation baselines, PocketGS achieves superior visual fidelity (LPIPS: 0.108) within a tight training budget (500 iterations, ∼4 minutes on an iPhone 15).

***Abstract*—While 3D Gaussian Splatting (3DGS) enables real-time rendering, its training demands workstation-level compute and memory, making mobile deployment impractical under minute-scale time budgets and limited peak memory. We present PocketGS, a mobile scene modeling paradigm that enables on-device 3DGS training under these tightly coupled constraints while preserving high-fidelity reconstruction. PocketGS resolves the fundamental tension between training efficiency, memory compactness, and modeling quality through three co-designed operators: $\mathcal{G}$ builds geometry-faithful point-cloud priors; $\mathcal{I}$ injects local surface statistics to seed anisotropic Gaussians, thereby reducing early conditioning gaps; and $\mathcal{T}$ unrolls alpha compositing with cached intermediates and index-mapped gradient scattering for stable mobile backpropagation. Extensive experiments demonstrate that PocketGS outperforms the powerful mainstream workstation 3DGS baseline under mobile budgets, delivering high-quality reconstructions and enabling a fully on-device, practical capture-to-rendering workflow.**

***Index Terms*—3D Gaussian Splatting, Rendering, on-devide, modeling system.**

I. Introduction

3D Gaussian Splatting (3DGS) [1] has emerged as a compelling paradigm for high-fidelity scene modeling, enabling real-time rendering across mixed reality, digital twins, and robotics. By replacing implicit neural fields [2] with explicit, rasterization-friendly primitives, 3DGS achieves a strong quality–efficiency balance, suggesting a natural next step: bringing 3D scene reconstruction fully onto commodity mobile devices for immediate, privacy-preserving, self-contained capture-to-render pipelines without cloud offloading.

Yet despite its inference efficiency, 3DGS remains fundamentally designed for resource-unconstrained training, assuming abundant compute, memory, and time via offline SfM [3] and long-horizon optimization. These assumptions break down on mobile devices, where strict runtime, memory, and thermal budgets define a different operating regime, leading to unstable optimization, excessive latency, and impractical memory usage when existing pipelines are ported directly.

Recent work targets isolated components of this pipeline, including faster optimization [4]–[6], mobile rendering and pruning [7]–[9], and large-scale deployment [10], while still relying on offline reconstruction or workstation-class resources. **To the best of our knowledge, no prior academic research has demonstrated a 3DGS training pipeline capable of running fully end-to-end on a mobile phone.**

We argue that on-device 3DGS is not an engineering question of efficiency, but requires rethinking the pipeline under tightly coupled resource constraints. Conventional 3DGS implicitly assumes decoupled stages of geometry recovery, initialization, and optimization, each enjoying abundant resources. Under mobile constraints, these stages become strongly interdependent, giving rise to three fundamental contradictions:

Input–Recovery Contradiction. Mobile captures yield

noisy poses and sparse geometry from on-device VIO [11], yet faithful 3DGS reconstruction needs reliable geometric priors. Existing methods compensate via aggressive densification [1], incurring substantial compute and memory overhead, so reducing this reliance demands more informative inputs.

Initialization–Convergence Contradiction. Standard 3DGS seeds isotropic Gaussians without exploiting local surface structure [1], requiring many iterations for meaningful geometry to emerge. Under strict iteration budgets such delayed structure formation is prohibitive, calling for initialization that improves optimization conditioning from the outset.

Hardware–Differentiability Contradiction. Differentiable splatting needs intermediate compositing states, whereas mobile GPUs expose only final blended outputs. Recovering these states via readback or recomputation introduces prohibitive bandwidth and memory costs, necessitating a hardware-aligned formulation of differentiable rendering.

These contradictions show that efficient on-device 3DGS cannot be obtained by optimizing components in isolation, and instead requires a *co-designed pipeline* that jointly addresses geometry, initialization, and optimization under shared resource constraints.

To this end, we present **PocketGS**, a unified framework for end-to-end 3DGS training directly on commodity smartphones. Given casually captured RGB images, PocketGS reconstructs a scene entirely on-device within minutes under strict memory budgets, via three tightly coupled operators. The geometry operator $\mathcal{G}$ constructs a compact yet reliable prior, reducing dependence on expensive densification. The initialization operator $\mathcal{I}$ leverages this prior to produce well-conditioned anisotropic Gaussians for efficient convergence under limited iterations. The training operator $\mathcal{T}$ reformulates differentiable splatting to align with mobile GPU execution, ensuring stable and bandwidth-efficient training.

Our main contributions are:

- The first publicly documented end-to-end 3D Gaussian Splatting training pipeline that runs fully on a commodity smartphone under strict runtime and memory constraints, enabling practical on-device capture-to-render workflows.
- Identification of three fundamental contradictions, namely input–recovery, initialization–convergence, and hardware–differentiability, that arise when adapting 3DGS to resource-constrained settings and must be addressed jointly rather than in isolation.
- A co-designed solution comprising (i) a compact geometry-prior construction that reduces reliance on costly densification, (ii) a prior-conditioned Gaussian parameterization that improves optimization conditioning, and (iii) a hardware-aligned differentiable splatting formulation enabling efficient gradient computation on mobile GPUs.

Experiments show that PocketGS achieves strong perceptual quality under strict mobile constraints, and remains competitive with or even surpasses workstation-based pipelines under matched budgets, suggesting that high-fidelity 3D reconstruction can be made practical on commodity devices.

II. Related Work

A. Novel View Synthesis and Gaussian Splatting

Novel view synthesis [12], [13] has evolved from implicit neural radiance fields [2], [14], [15] to explicit grid- and hash-based representations [16]–[20], and recently to 3D Gaussian Splatting [1], [21], [22], which enables real-time rendering through rasterization-friendly primitives. Subsequent 3DGS variants target reconstruction quality, SLAM integration, geometry-aware modeling, and optimization [5], [23]–[29]. A remaining bottleneck is that most pipelines still depend on offline Structure-from-Motion [3], [30], which is costly and brittle on mobile-grade inputs.

B. Efficient and Mobile Gaussian Splatting

Efforts toward efficient 3DGS focus on training and memory reduction via pruning or adaptive optimization [4], [31], and on mobile rendering via foveation or GPU-aware kernels [8]. Larger-scale mobile deployments and perception extensions have also been demonstrated [10], [32]. Orthogonally, mobile 3D scanning stacks based on VIO and MVS [11], [33]–[37] provide only sparse geometry, while early neural-rendering ports to mobile [38], [39] target inference or cloud-assisted processing rather than training. Closing this gap, namely the lack of a complete capture-to-model 3DGS pipeline that runs fully on-device under mobile budgets, is the problem PocketGS addresses.

III. The PocketGS Paradigm

PocketGS enables end-to-end on-device 3DGS training under strict resource constraints, for static scene reconstruction from handheld captures. The input is an RGB sequence $\{I_t\}_{t=1}^{n}$ with coarse poses $\{\hat{T}_t\}_{t=1}^{n}$ from a mobile tracker (e.g., ARKit, ARCore); the output is an optimized Gaussian set Θ^* for novel view synthesis.

PocketGS comprises three tightly coupled operators: $\mathcal{G}$ refines $\{\hat{T}_t\}$ into accurate $\{T_t\}$ and extracts a dense point cloud P as a geometric prior, $\mathcal{I}$ maps P to a well-conditioned Θ_0, and $\mathcal{T}$ performs hardware-aligned differentiable rendering to optimize Θ_0 into Θ^*. Formally, $\mathcal{G} : (\{I_t, \hat{T}_t\}) \mapsto (\{T_t\}, P)$, $\mathcal{I} : P \mapsto \Theta_0$, and $\mathcal{T} : (\Theta_0, \{I_t, T_t\}) \mapsto \Theta^*$.

A. Geometry Prior Construction under Resource Budgets

Standard 3DGS pipelines rely on prolonged optimization with training-time densification to recover structure from noisy, sparse inputs, which on mobile devices yields unstable geometry and excessive cost. PocketGS instead builds a dense, low-noise prior directly from $\{I_t, \hat{T}_t\}$ via $\mathcal{G}$; conditioning optimization on refined $\{T_t\}$ and a compact P reduces in-loop densification and improves stability.

1) Information-Gated Frame Subsampling: Handheld captures suffer from motion blur, defocus, and temporal redundancy. We therefore retain only sharp, geometrically useful frames via a two-stage information gate.

Displacement Gate. Let $\mathbf{t}_{curr}$ and $\mathbf{t}_{last}$ be the camera centers of the current and last selected keyframes. A frame is kept only if

$$d = \|\mathbf{t}_{curr} - \mathbf{t}_{last}\|_2 \;\geq\; \tau_d,$$

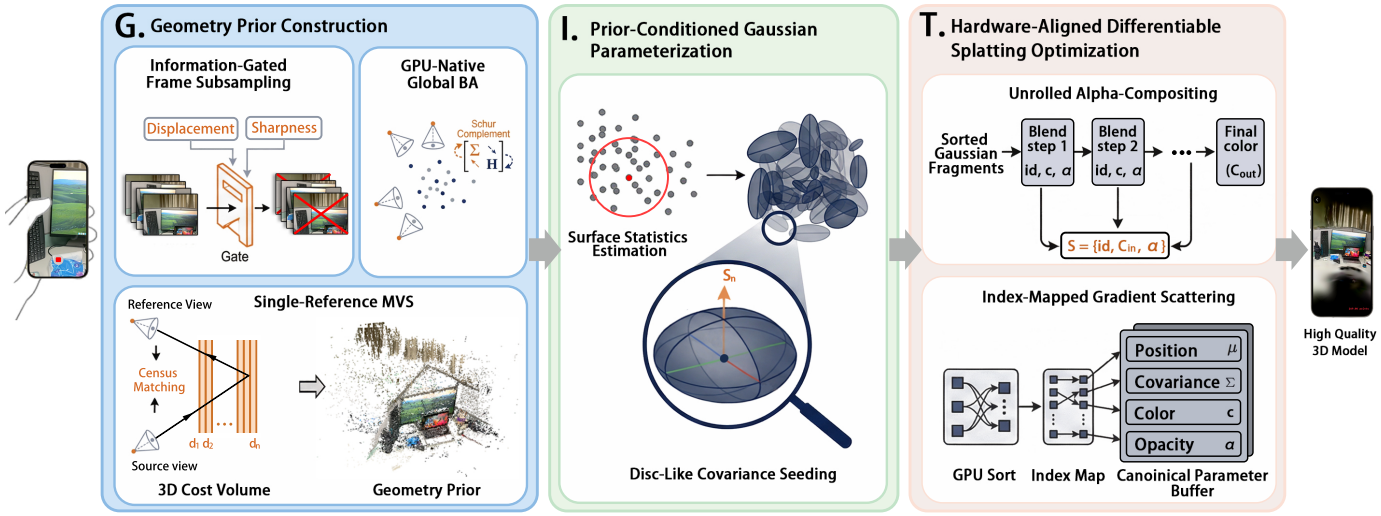

Fig. 2: **Overview of the PocketGS framework.** PocketGS tackles on-device 3DGS training through three coupled operators: ($\mathcal{G}$) Geometry Prior Construction employs an information-gated gate for frame selection, followed by GPU-native Schur-complement BA and single-reference MVS. The MVS module constructs a 3D cost volume by sampling depth hypotheses $d_1, \ldots, d_n$ via census matching to produce a dense geometric scaffold. ($\mathcal{I}$) Prior-Conditioned Parameterization seeds anisotropic Gaussians by estimating local surface statistics (normals $\mathbf{s}_n$ and scales) to front-load structure discovery via disc-like covariance seeding. ($\mathcal{T}$) Hardware-Aligned Splatting implements a mobile-native differentiable renderer using unrolled alpha-compositing ($S = id, C_{in}, \alpha$) and index-mapped gradient scattering to ensure stable backpropagation within tight mobile memory bounds of the canonical parameter buffer (μ, Σ, c, α).

with $\tau_d = 0.05\,\mathrm{m}$, which suppresses redundant viewpoints while preserving parallax for triangulation.

Sharpness Heuristic. For frames admitted by the displacement gate, we reject blurred ones via gradient energy on a sparse luma grid Ω with stride Δ:

$$S = \frac{1}{|\Omega|} \sum_{(x,y)\in\Omega} \left(|I(x+\Delta, y) - I(x,y)| + |I(x, y+\Delta) - I(x,y)| \right), \tag{1}$$

which correlates with high-frequency content capture and favors sharper frames.

Candidate Windowing. We keep a sliding window of 8 eligible frames and retain a single representative, replacing the current best S_{best} by a candidate S_{new} only if $S_{new} > S_{best}$.

2) GPU-Native Global BA as Mobile MAP Refinement: Mobile tracking poses are typically too noisy for 3DGS. We therefore run a GPU-native global BA initialized from $\{\hat{T}_i\}$ to jointly refine poses and sparse 3D points, with robustified reprojection objective

$$\min_{\{T_i\},\{P_j\}} \sum_{(i,j)\in\mathcal{O}} \rho\Big(\big\| \pi(T_i, P_j) - \mathbf{p}_{ij} \big\|^2_{W_{ij}} \Big), \tag{2}$$

where π is the camera projection, $T_i \in SE(3)$ and $P_j \in \mathbb{R}^3$ the pose and point parameters, $\mathbf{p}_{ij} \in \mathbb{R}^2$ the measured image location, $W_{ij} = \Sigma_{ij}^{-1}$ the information weighting, $\mathcal{O}$ the valid-observation set, and $\rho(\cdot)$ the Huber loss.

Scale-Aware Gauge Fixing. To resolve the 7-DoF gauge ambiguity, we fix the first keyframe's 6-DoF pose and constrain the second keyframe's translation along the dominant baseline axis, fixing the global scale and yielding a stable metric frame.

GPU-Native Schur Complement.

The BA bottleneck is the normal equation $\mathbf{H}\Delta = \mathbf{b}$ with $\mathbf{H} = \mathbf{J}^\top\mathbf{J}$. Partitioning Δ into pose (t) and point (p) blocks yields the reduced camera system

$$(\mathbf{H}_{tt} - \mathbf{H}_{tp}\mathbf{H}_{pp}^{-1}\mathbf{H}_{pt})\Delta_t = \mathbf{b}_t - \mathbf{H}_{tp}\mathbf{H}_{pp}^{-1}\mathbf{b}_p. \tag{3}$$

Since $\mathbf{H}_{pp}$ is block-diagonal with independent 3×3 blocks, $\mathbf{H}_{pp}^{-1}$ is computed fully in parallel on the GPU, and Δ_p is recovered by back-substitution from Δ_t.

Iterative Geometric Refinement. BA is embedded in a refinement loop: after each global update we re-triangulate features with the updated poses, drop points with high reprojection error or insufficient triangulation angle, and discard behind-camera observations, purifying the sparse scaffold for MVS.

3) Single-Reference Cost-Volume MVS: Mobile tracking clouds are too sparse to initialize 3DGS reliably. We address this with a lightweight MVS stage that recovers a denser, geometry-faithful cloud under mobile budgets.

Probabilistic Depth Range Estimation. Plane-sweep efficiency hinges on the depth range. For each target frame we project sparse BA points, take the depth bounds q_1, q_2, and set the search range to $[q_{.05}\cdot 0.5,\ q_{.95}\cdot 1.5]$, intentionally widened to tolerate BA noise while focusing on the most probable scene volume.

Memory-Efficient Cost Volume. To minimize memory, each target view uses a single reference, chosen to maximize

$$S_{ref} = \exp\left(-\frac{(b - b_{target})^2}{2\sigma_b^2} \right) \cdot \max\left(\frac{\alpha}{\alpha_{min}}, 1 \right), \tag{4}$$

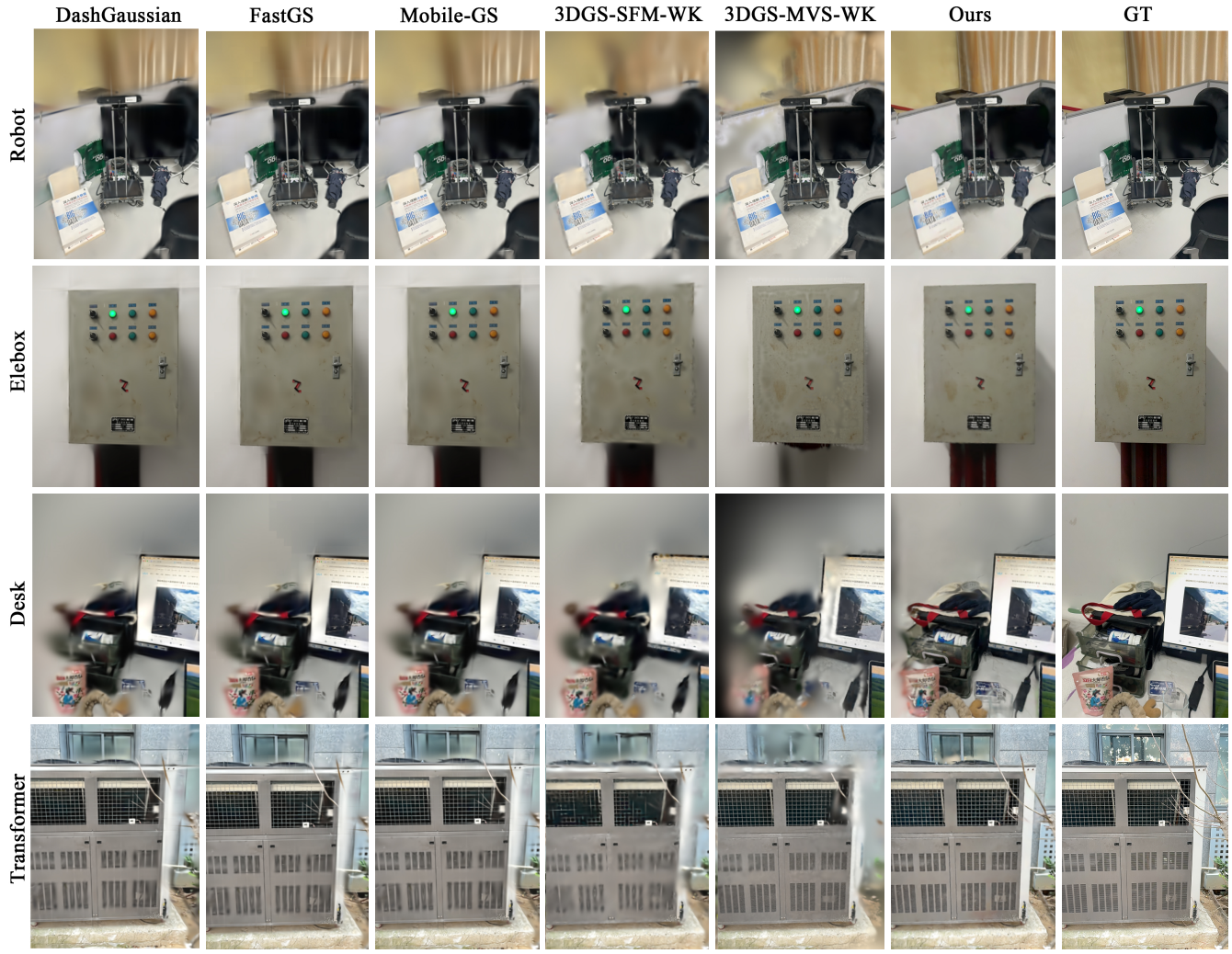

Fig. 3: Qualitative comparison on MobileScan under the matched mobile-budget protocol. PocketGS recovers sharper textures and finer details than the workstation baselines. 3DGS-MVS-WK still exhibits artifacts despite a comparably dense prior because its MVS-confidence-weighted distribution is *spatially biased toward textured regions*, leaving low-texture and specular structure under-sampled at initialization—a deficit a 500-iteration budget cannot redistribute around. PocketGS's surface-aligned, spatially balanced initialization yields higher structural fidelity closer to ground truth (GT).

where b is the candidate baseline, b_{target} the preferred baseline, and α the viewing angle, favoring informative parallax without overly wide baselines that hurt photometric consistency.

Matching cost uses the Census Transform [40] for illumination robustness and plane-sweep depth [41] with SGM aggregation [42]. Depth maps are filtered at confidence 0.4 (a robust density–outlier trade-off) and fused into P, which scaffolds $\mathcal{I}$.

B. Prior-Conditioned Gaussian Parameterization

Standard 3DGS initializes all points as isotropic Gaussians from inter-point distances alone, ignoring local surface geometry and forcing heavy parameter restructuring during optimization—prohibitive under tight iteration budgets. PocketGS instead uses P to seed surface-aligned anisotropic Gaussians via $\mathcal{I}$, giving a geometry-consistent starting state that ccelerates on-device convergence.

1) Local Surface Statistics Estimation: For each $\mathbf{p}_i$ in the dense cloud we estimate surface statistics from two KNN neighborhoods of deliberately different sizes: $K_n = 16$ for the normal (a larger neighborhood suppresses sampling noise) and $K_s = 3$ for the tangential scale which must track local density to avoid over-smoothing thin structures. Using $\{\mathbf{p}_{i,k}\}_{k=1}^{K_n}$ with centroid $\bar{\mathbf{p}}_i = \frac{1}{K_n}\sum_{k=1}^{K_n}\mathbf{p}_{i,k}$, the local covariance is

$$\mathbf{C}_i = \frac{1}{K_n}\sum_{k=1}^{K_n}(\mathbf{p}_{i,k}-\bar{\mathbf{p}}_i)(\mathbf{p}_{i,k}-\bar{\mathbf{p}}_i)^{\mathsf{T}}, \tag{5}$$

whose smallest-eigenvalue eigenvector gives the local normal $\mathbf{n}_i$. This fixed-K scheme is embarrassingly parallel and memory-bounded, fitting mobile GPUs naturally.

2) Disc-Like Covariance Seeding: Each Gaussian is seeded as a thin disc-like ellipsoid tangent to the local surface, injecting geometry-aware anisotropy directly into the param-

Method	LLFF					Mip-NeRF 360					MobileScan				
	PSNR↑	SSIM↑	LPIPS↓	Time↓(s)	Count	PSNR↑	SSIM↑	LPIPS↓	Time↓(s)	Count	PSNR↑	SSIM↑	LPIPS↓	Time↓(s)	Count
3DGS-SFM-WK	23.28	0.789	0.226	116.3	18k	21.05	0.649	0.377	398.7	46k	22.64	0.781	0.319	123.8	23k
3DGS-MVS-WK	21.04	0.730	0.262	320.3	41k	18.26	0.586	0.429	975.6	76k	22.23	**0.834**	0.226	547.5	165k
Mobile-GS	23.26	0.790	0.227	133.1	18k	21.06	0.650	0.377	405.2	46k	22.61	0.782	0.321	145.1	23k
DashGaussian	23.30	0.790	0.224	115.9	18k	21.54	0.672	0.351	399.9	46k	22.66	0.783	0.317	**122.5**	23k
taming-3dgs	23.27	0.788	0.228	117.6	18k	21.09	0.651	0.375	402.6	46k	22.62	0.780	0.322	125.1	23k
FastGS	23.31	0.788	0.227	116.3	18k	21.12	0.652	0.373	396.4	46k	22.63	0.780	0.320	123.9	23k
speedy-splat	23.29	0.789	0.225	133.1	18k	21.07	0.650	0.376	425.8	46k	22.65	0.781	0.319	144.5	23k
PocketGS (Ours)	**23.54**	**0.791**	**0.222**	**105.4**	33k	**22.25**	**0.702**	**0.302**	**145.5**	77k	**23.67**	0.791	**0.225**	255.2	168k

TABLE I: **Average metrics across datasets.** Dataset-level averages on LLFF, Mip-NeRF 360 and MobileScan. Higher is better for PSNR/SSIM, and lower is better for LPIPS/Time. Red, orange, and yellow denote the best, second-best, and third-best results, respectively.

eterization. The tangential scale is the average distance to the K_s nearest neighbors,

$$s_t = \frac{1}{K_s} \sum_{k=1}^{K_s} \|\mathbf{p}_i - \mathbf{p}_{i,k}\|, \tag{6}$$

and the normal-direction scale is $s_n = r_{normal}\, s_t$, giving a thin surface-aligned primitive with enough thickness for stable training. Scales are optimized in log-space for positivity; the initial rotation $\mathbf{q}_i$ aligns the Gaussian's z-axis with $\mathbf{n}_i$, and opacity is initialized to the logit of 0.1. Estimated normals serve only for initialization, never as training constraints; their role is a better-conditioned $\Theta_0 = \mathcal{I}(P)$, which matters most under strict iteration budgets.

C. Hardware-Aligned Differentiable Splatting Optimization

On-device 3DGS training is bandwidth-bound on mobile GPUs: fixed-function blending exposes only the final composited color, yet backpropagation needs intermediate alpha-compositing states, and materializing large auxiliary buffers incurs heavy memory traffic. We therefore unroll front-to-back compositing into a differentiable operator and cache a compact forward replay trace, yielding a GPU-resident map $\mathcal{T} : (\Theta_0, \{I_t, T_t\}) \mapsto \Theta^*$ that produces correct gradients without framebuffer readbacks or backward-time reconstruction of per-pixel splat sequences.

1) Unrolled Alpha-Compositing with Forward Replay Cache: Fixed-function blending exposes only the final composited color, leaving fragment identities and depth order inaccessible; reconstructing them during backprop is prohibitive.We therefore (1) unroll front-to-back compositing into an explicit computation graph and (2) cache a compact forward replay trace per pixel.

a) Explicit Computation Graph.: For depth-sorted visible Gaussians, we explicitly compose

$$C_{out} = C_{in}(1-\alpha) + \alpha\, c, \tag{7}$$

where C_{in} is the incoming accumulator and (α, c) are the fragment's opacity and color. Backprop follows the same unrolled chain, yielding closed-form gradients for all three quantities:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial C_{in}} &= \frac{\partial \mathcal{L}}{\partial C_{out}}\,(1-\alpha), \quad \frac{\partial \mathcal{L}}{\partial c} = \alpha\, \frac{\partial \mathcal{L}}{\partial C_{out}}, \\ \frac{\partial \mathcal{L}}{\partial \alpha} &= \Big\langle \frac{\partial \mathcal{L}}{\partial C_{out}},\; c - C_{in} \Big\rangle, \end{aligned} \tag{8}$$

where $\langle \cdot, \cdot \rangle$ is the per-channel inner product. These gradients are then scattered to the canonical Gaussian parameter buffer via the index map of Sec. III-C2.

b) Bandwidth-Efficient Forward Replay Cache.: To avoid reconstructing per-pixel splat sequences during backprop, we record only the replay-essential quantities

$$S = \{\mathrm{id},\, C_{in},\, \alpha\}, \tag{9}$$

where id is the sorted splat identifier used to scatter gradients. A per-pixel counter $\mathrm{count}(u)$ tracks valid entries. Each iteration resets only the $O(WH)$ counters rather than the full $O(WHK_{\max})$ cache, and backprop traverses only $k \in [0, \mathrm{count}(u) - 1]$. Because the backward shader indexes solely inside this valid range, stale entries outside it are never read and cannot corrupt gradients; counter reset therefore preserves exact gradient correctness while avoiding the bandwidth-heavy cache clear.

2) Index-Mapped Gradient Scattering: Depth-sorted rendering permutes Gaussians, and physically reordering parameter buffers would incur heavy data movement and risk misaligning optimizer states. We therefore decouple the *sorted view* (rendering/backprop) from the *canonical layout* (parameters and optimizer moments). A GPU sort yields a permutation π mapping sorted index i to canonical index $\pi(i)$; kernels operate on the sorted view while gradients are scattered back as

$$\nabla\theta_{\pi(i)} \mathrel{+}= g_i, \tag{10}$$

with g_i the gradient of the i-th sorted element, keeping Adam states aligned with their parameters without CPU intervention or redundant memory copies.

IV. Experiments

a) Platform and budget.: PocketGS is implemented in Swift on Apple Metal and runs entirely on an iPhone 15 (Apple A16, 6 GB unified memory). All methods share identical resolution, color space, metric implementations, and a matched 500-iteration budget. This budget reflects both hardware constraints and realistic mobile usage patterns. On mobile devices, extended training sessions lead to thermal throttling and

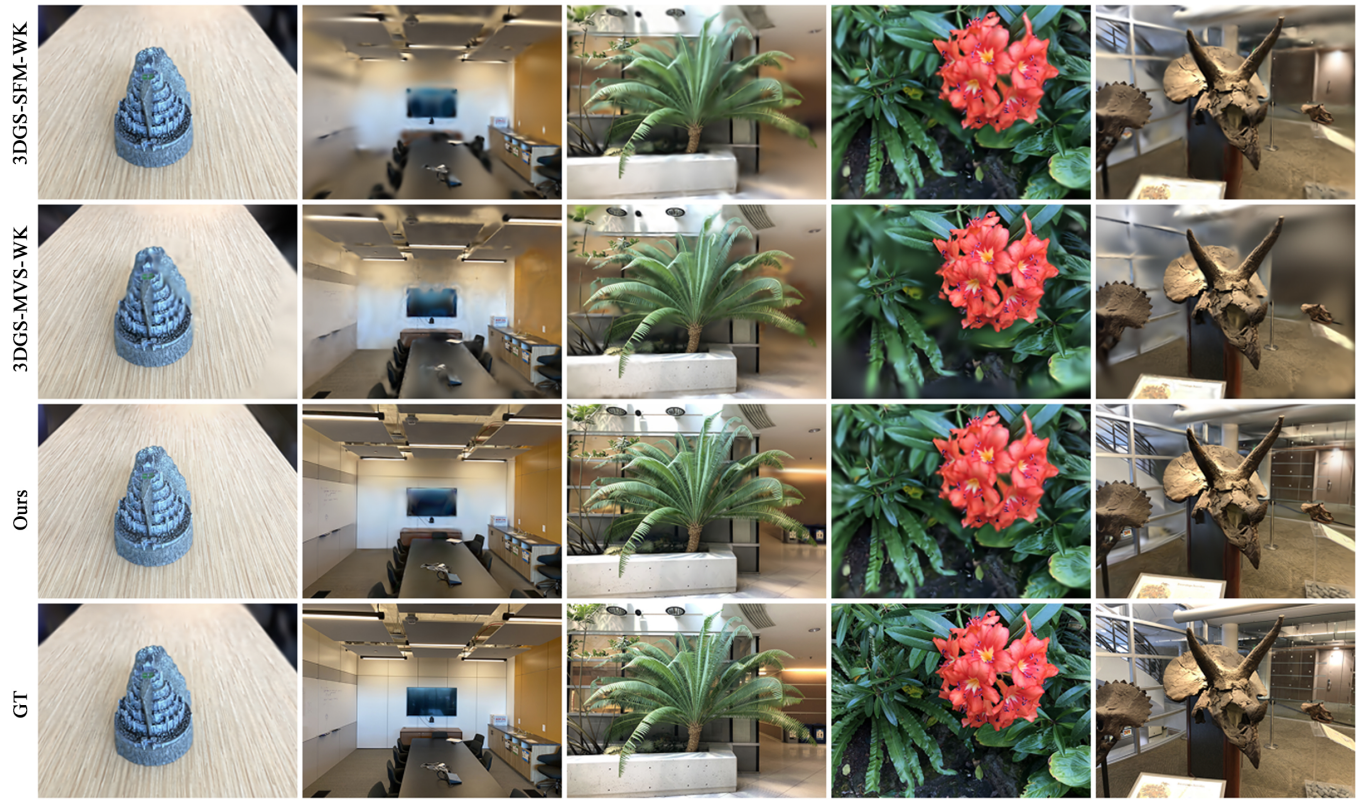

Fig. 4: **Qualitative comparison on the LLFF dataset.** Our method produces much sharper textures and more accurate thin structures (e.g., the leaves in Fern and the petals in Flower), closely matching the ground truth (GT).

significant battery drain, limiting the practical window to 3–5 minutes. Additionally, typical users expect rapid turnaround rather than prolonged wait times. By contrast, workstation-based 3DGS training often requires 7k–30k iterations, far exceeding what users tolerate on mobile devices.

b) Baselines.: We compare against two families of baselines. The *geometry-prior* family swaps priors fed into a vanilla 3DGS optimizer: *3DGS-SFM-WK* uses COLMAP SfM [3], while *3DGS-MVS-WK* uses dense COLMAP MVS. The *training-side* family fixes the SfM prior and varies the optimizer: Mobile-GS [8], DashGaussian [43], Taming-3DGS [4], FastGS [44], and speedy-splat [44]. Workstation runs use two NVIDIA RTX 3090 GPUs. PocketGS occupies a third corner where prior construction and prior-conditioned initialization are co-designed.

c) Datasets and metrics.: We evaluate on Mip-NeRF 360 [15], LLFF [45], and our phone-captured MobileScan, using each dataset's standard split (consistent with the 3DGS protocol [46]) for all methods. We report PSNR, SSIM, LPIPS, and end-to-end runtime $T_{\text{total}} = T_{\text{geom}} + T_{\text{train}}$, and Gaussian count. Full implementation details, dataset descriptions, and extended experiments up to 1000 iterations are in the supplement.

A. Main Results

Table I reports the averages under the mobile-budget protocol, and Fig. 3 shows qualitative comparisons on MobileScan. PocketGS attains the best overall quality–efficiency trade-off across all three datasets, while keeping a compact or comparable Gaussian count. This matches our central hypothesis: under short optimization budgets, prior conditioning dominates final quality, and improving the prior is more effective than accelerating an optimizer that consumes a fragile input. At their native workstation budgets the SfM-based baselines reach substantially higher absolute fidelity; such unconstrained-budget comparisons are orthogonal to our contribution, since none of those configurations fit the on-device time and thermal envelope this work targets.

a) LLFF: On forward-facing real captures where imperfect SfM destabilizes early optimization, PocketGS tops every dataset-level metric, with 23.54 dB PSNR, 0.791 SSIM and 0.222 LPIPS, and is also the fastest at 105.4 s. The dense workstation pipeline 3DGS-MVS-WK drops to 21.04 dB despite using 41k Gaussians: under a mobile budget, a larger unconditioned point cloud inflates per-iteration cost without yielding well-conditioned gradients, slowing rather than aiding convergence. PocketGS reaches higher fidelity with only 33k Gaussians, evidence that prior-conditioned initialization yields a better-conditioned landscape under short budgets. The five training-side accelerators cluster tightly around SFM-WK since they consume the same sparse geometry; their reported gains materialize at much longer horizons and largely vanish at 500 iterations, where the prior is the bottleneck.

b) Mip-NeRF 360.: Mip-NeRF 360 stresses the unbounded indoor and outdoor regime with wide baselines and strong view-dependent effects, where the 500-iteration budget

Fig. 5: **Qualitative results on a mobile device across diverse datasets.** We showcase real-time rendering screenshots from a mobile Phone, covering our self-collected scenes (rows 1-2), LLFF (row 3), and Mip-NeRF 360 (row 4). PocketGS consistently achieves high-fidelity reconstruction and sharp details across varying scene scales and capture conditions. The red labels indicate optimized point counts, demonstrating our method's representation efficiency and robust generalization for practical on-device deployment.

is far below what these scenes typically require. PocketGS still dominates every dataset-level metric, reaching 22.25 dB PSNR, 0.702 SSIM and 0.302 LPIPS at only 145.5 s, while every SfM-based baseline—including the five training-side accelerators and the dense workstation pipeline—needs ∼400 s or more to converge to clearly weaker quality. DashGaussian is the strongest competitor at 21.54 dB, 0.672 SSIM and 0.351 LPIPS, yet PocketGS surpasses it by 0.71 dB PSNR, 0.030 SSIM and 0.049 LPIPS while running 2.7× faster. The dense pipeline 3DGS-MVS-WK collapses to 18.26 dB at 975.6 s: its generic MVS prior is overwhelmed by unbounded backgrounds and complex parallax, and the resulting noisy initialization cannot be recovered within the short budget. The five training-side accelerators again cluster tightly around the sparse SfM baseline (∼21.0–21.1 dB, ∼0.65 SSIM, ∼0.37 LPIPS), all ∼2.7–2.9× slower than PocketGS, confirming that on 360° unbounded scenes the bottleneck is the prior rather than the optimizer. PocketGS's surface-aligned, spatially balanced initialization produces a Gaussian set (77k) that already covers both near foreground and far background structure, letting the short optimization phase focus on appearance refinement instead of geometry recovery.

c) MobileScan.: MobileScan is the most challenging benchmark, since sparse, noisy and unevenly covered captures expose any pipeline that relies on a brittle prior. PocketGS reaches the top PSNR of 23.67 dB and the top LPIPS of 0.225 among all eight methods, gaining 1.01 dB over the strongest training-side baseline DashGaussian and 1.44 dB over 3DGS-MVS-WK while being more than 2× faster end-to-end, 255.2 s versus 547.5 s. 3DGS-MVS-WK retains the top SSIM of 0.834 thanks to its large 165k Gaussian count that smooths out residuals, yet its 547.5 s runtime is impractical on device. Fig. 3 reveals two failure modes that PocketGS avoids: SfM-only baselines show blurry textures and partial structural collapse from sparse initialization, while 3DGS-MVS-WK's prior, though comparable to ours in count, is *spatially biased toward textured regions* from MVS confidence weighting, leaving specular and thin structures under-sampled—a deficit 500 iterations cannot redistribute. PocketGS's surface-aligned, spatially balanced anisotropic initialization produces sharper textures that visually approach ground truth.

Overall, PocketGS delivers the best quality–efficiency trade-off across all three datasets, dominating both vanilla 3DGS and training-side accelerators on real forward-facing captures (LLFF), unbounded 360° scenes (Mip-NeRF 360) and noisy mobile captures (MobileScan). Training-side accelerators reduce wall-clock on the same SfM input but cannot recover the quality lost when the prior is fragile, which is precisely the

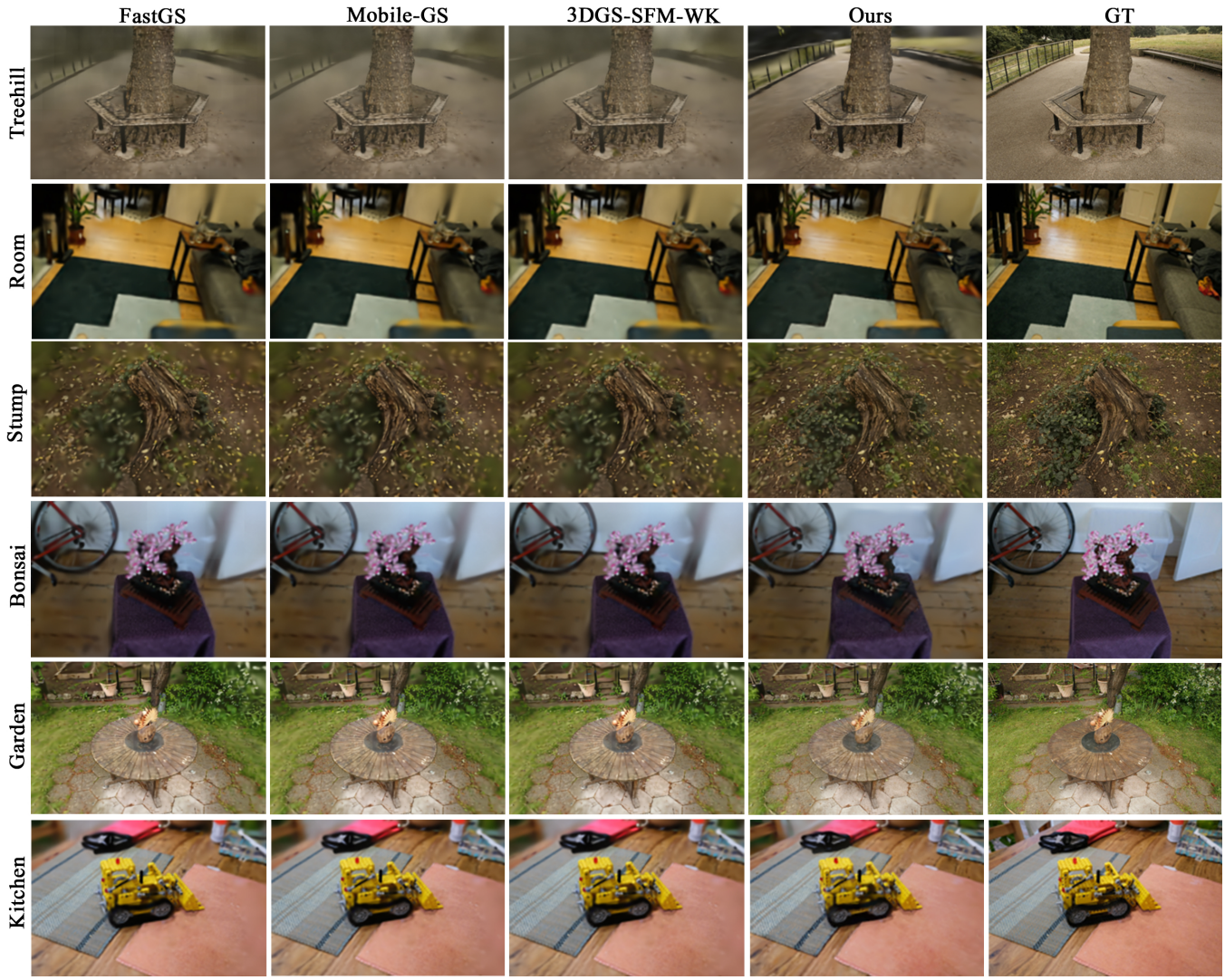

Fig. 6: **Qualitative comparison on the Mip-NeRF 360 dataset.** In these object-centric synthetic scenes, our method produces high-fidelity results.

regime our design targets.

B. On-device Rendering and Memory

a) On-device rendering.: Fig. 5 shows real-time rendering screenshots on a mobile Phone across MobileScan, LLFF, and Mip-NeRF 360 scenes; detail stays sharp across scales and capture conditions. Table II reports rendering throughput on the same device. PocketGS is the only system in the comparison that trains fully on-device yet still sustains real-time rendering, demonstrating that on-device training does not preclude efficient mobile deployment.

b) Memory footprint.: Peak memory stays within a practical on-device budget. Geometry prior construction averages 1.48 GB and ranges from 1.19 to 1.73 GB, while full 3DGS training averages 2.22 GB and ranges from 1.82 to 2.65 GB. Across all default MobileScan scenes the peak stays below 3 GB, which leaves clear headroom under the Phone's 6 GB unified memory.

Budget-sweep curves, early-convergence behavior, per-scene variance, and detailed memory footprints further supporting our method are provided in the supplement.

C. Ablation Study

We quantify the contribution of each PocketGS component on MobileScan under the matched 500-iteration protocol, with dataset-level averages summarized in Table III. Disabling prior-conditioned initialization $\mathcal{I}$ degrades every metric and extends end-to-end training time from 255.2 s to 317.0 s, confirming that surface-aligned anisotropic seeding is critical for convergence *efficiency* under a strict iteration budget, not merely for asymptotic fidelity. Disabling the GPU-native global BA leaves PSNR and runtime nearly unchanged but drops SSIM from 0.791 to 0.760, indicating that BA contributes mainly to structural coherence at negligible cost. Disabling the lightweight single-reference MVS produces the largest quality degradation, with PSNR falling to 21.15 and LPIPS rising to 0.400. Runtime drops to 127.1 s because the

Fig. 7: Ablation of information-gated frame subsampling. The full pipeline preserves sharper details by filtering motion- or defocus-degraded frames before geometry prior construction.

Method	On-device Training	Render FPS↑
MobileNeRF [38]	×	31.40
MetaSapiens [7]	×	49.24
Mobile-GS [8]	×	62.56
PocketGS (Ours)	✓	52.32

TABLE II: **Deployment-side rendering on iPhone 15.** Baselines are rendering-only references with offline model preparation; PocketGS additionally trains fully on-device.

optimizer now sees a sparse prior of about 23k points instead of the dense 168k; yet the -2.52 dB PSNR loss is far steeper than the roughly 50% time saving, confirming that under 500 iterations the dense MVS prior is essential for fidelity, not merely for runtime amortization. Qualitatively, as shown in Fig. 7, disabling information-gated frame subsampling admits motion- and defocus-degraded frames that corrupt the geometry prior and cascade into blurrier reconstructions.

a) Hardware-aligned operator $\mathcal{T}$.: Replacing our replay-cache plus per-pixel counter reset with simpler alternatives either breaks mobile feasibility or wastes bandwidth: the CPU-readback variant pushes peak memory to 3.84 GB, exceeding the on-device budget, while the full-cache-clear variant evicts entries that backward replay never revisits. Our design preserves gradient correctness while reducing backward time and peak memory on mobile, and differs from the recompute-based backward of the original 3DGS implementation [46], which we evaluate as the *Naive Recompute* variant in Sec. E of the supplement.

b) Geometry prior vs. COLMAP: On MobileScan, PocketGS geometry construction combines GPU-native BA with lightweight MVS, producing a prior of comparable density to COLMAP SfM+MVS while cutting geometry time from 619.8 s to 31.4 s, a roughly $19.8\times$ speedup. This supports our claim that the mobile geometry stage is not a trimmed COLMAP but a distinct operating point co-designed with the subsequent initialization and training operators.

Per-scene breakdowns, the full $\mathcal{T}$-variant study with gradient-error and memory/latency numbers, runtime decomposition, confidence-threshold sensitivity, and memory footprint are reported in the supplement materials.

Variant	PSNR↑	SSIM↑	LPIPS↓	Time↓ (s)
PocketGS (Full)	**23.67**	**0.791**	**0.225**	255.2
w/o Initialization $\mathcal{I}$	22.49	0.778	0.249	317.0
w/o Global BA	23.54	0.760	0.227	251.8
w/o MVS	21.15	0.667	0.400	**127.1**

TABLE III: **Ablation on MobileScan.** Dataset-level averages over the 16 scenes under the matched 500-iteration protocol. The runtime drop in the w/o MVS row reflects the smaller initial primitive count rather than a favorable quality–efficiency trade-off, as discussed in the text.

V. Conclusion

In this paper, we present **PocketGS**, a fully on-device training framework for 3D Gaussian Splatting that targets the practical constraints of mobile devices. PocketGS is built upon three co-designed operators: **(G)** geometry-prior construction to provide compact and reliable geometric guidance, **(I)** prior-conditioned Gaussian parameterization to improve initialization and optimization under iteration-limited budgets, and **(T)** hardware-aligned splatting optimization to ensure efficient and stable differentiable splatting on mobile GPUs. Extensive experiments on public benchmarks and our MobileScan dataset show that PocketGS achieves strong perceptual quality while meeting smartphone budgets, e.g., **approximately 4-minute** end-to-end training with **<3 GB** peak memory on iPhone 15. These results indicate that high-fidelity 3D reconstruction and mobile 3D content creation can be made practical on commodity devices.

References

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, 2023.

[2] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.

[3] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] S. S. Mallick, R. Goel, B. Kerbl, M. Steinberger, F. V. Carrasco, and F. De La Torre, "Taming 3dgs: High-quality radiance fields with limited resources," in *SIGGRAPH Asia 2024 Conference Papers*, 2024, pp. 1–11.

[5] Y. Chen, J. Jiang, K. Jiang, X. Tang, Z. Li, X. Liu, and Y. Nie, "Dashgaussian: Optimizing 3d gaussian splatting in 200 seconds," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 11 146–11 155.

[6] A. Hanson, A. Tu, G. Lin, V. Singla, M. Zwicker, and T. Goldstein, "Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 21 537–21 546.

[7] W. Lin, Y. Feng, and Y. Zhu, "Metasapiens: Real-time neural rendering with efficiency-aware pruning and accelerated foveated rendering," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 669–682.

[8] X. Du, Y. Wang, K. Zhan, and X. Yu, "Mobile-gs: Real-time gaussian splatting for mobile devices," 2026. [Online]. Available: https://arxiv.org/abs/2603.11531

[9] M. M. R. Sanim, Z. Shu, B. Afsharmanesh, A. Mirian, J. Guan, W. Niu, B. Ren, and G. Agrawal, "Optimizing 3d gaussian splattering for mobile gpus," in *2025 34th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 2025, pp. 359–371.

[10] Z. Liu, H. Zhu, X. Li, Y. Wang, Y. Shi, W. Li, J. Leng, M. Guo, and Y. Feng, "Voyager: Real-time splatting city-scale 3d gaussians on your phone," *arXiv e-prints*, pp. arXiv–2506, 2025.

[11] T. Qin, P. Li, and S. Shen, "Vins-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.

[12] A. Tewari, J. Thies, B. Mildenhall, P. Srinivasan, E. Tretschk, W. Yifan, C. Lassner, V. Sitzmann, R. Martin-Brualla, S. Lombardi *et al.*, "Advances in neural rendering," in *Computer Graphics Forum*, vol. 41, no. 2. Wiley Online Library, 2022, pp. 703–735.

[13] K. Gao, Y. Gao, H. He, D. Lu, L. Xu, and J. Li, "Nerf: Neural radiance field in 3d vision, a comprehensive review," *arXiv preprint arXiv:2210.00379*, 2022.

[14] J. T. Barron, B. Mildenhall, M. Tancik, P. Hedman, R. Martin-Brualla, and P. P. Srinivasan, "Mip-NeRF: A Multiscale Representation for Anti-Aliasing Neural Radiance Fields," in *ICCV*, 2021.

[15] J. T. Barron, B. Mildenhall, M. Tancik, P. P. Srinivasan, X. Han, and R. Martin-Brualla, "Mipnerf 360: Unbounded anti-aliased neural radiance fields," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[16] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa, "Plenoxels: Radiance fields without neural networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 5501–5510.

[17] C. Sun, M. Sun, and H.-T. Chen, "Direct Voxel Grid Optimization: Super-Fast Convergence for Radiance Fields Reconstruction," in *CVPR*, 2022.

[18] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Trans. Graph.*, vol. 41, no. 4, 2022.

[19] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Zip-NeRF: Anti-Aliased Grid-Based Neural Radiance Fields," in *ICCV*, 2023.

[20] W. Guo, B. Wang, and L. Chen, "Neuv-slam: Fast neural multiresolution voxel optimization for rgbd dense slam," *IEEE Transactions on Multimedia*, 2025.

[21] G. Fang and B. Wang, "Mini-splatting: Representing scenes with a constrained number of gaussians," in *European Conference on Computer Vision*. Springer, 2024, pp. 165–181.

[22] ——, "Mini-splatting2: Building 360 scenes within minutes via aggressive gaussian densification," *arXiv preprint arXiv:2411.12788*, 2024.

[23] Z. Peng, T. Shao, Y. Liu, J. Zhou, Y. Yang, J. Wang, and K. Zhou, "Rtg-slam: Real-time 3d reconstruction at scale using gaussian splatting," in *ACM SIGGRAPH 2024 conference papers*, 2024, pp. 1–11.

[24] C. Yan, D. Qu, D. Xu, B. Zhao, Z. Wang, D. Wang, and X. Li, "Gs-slam: Dense visual slam with 3d gaussian splatting," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[25] G. Wenzhi, B. Haiyang, M. Yuanqu, L. Jia, and C. Lijun, "Fvloc-nerf: Fast vision-only localization within neural radiation field," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 3329–3334.

[26] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, "2d gaussian splatting for geometrically accurate radiance fields," *ACM SIGGRAPH 2024 Conference Proceedings*, 2024.

[27] D. Chen, H. Li, W. Ye, Y. Wang, W. Xie, S. Zhai, N. Wang, H. Liu, H. Bao, and G. Zhang, "PGSR: Planar-based Gaussian Splatting for Efficient and High-Fidelity Surface Reconstruction," *IEEE TVCG*, 2024.

[28] L. Höllein, A. Božič, M. Zollhöfer, and M. Nießner, "3dgs-lm: Faster gaussian-splatting optimization with levenberg-marquardt," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 26 740–26 750.

[29] G. Fang and B. Wang, "Efficient Scene Modeling Via Structure-Aware and Region-Prioritized 3D Gaussians," *IEEE TPAMI*, 2025.

[30] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon, "Bundle adjustment — a modern synthesis," in *Vision Algorithms: Theory and Practice*. Springer, 1999.

[31] L. Lan, T. Shao, Z. Lu, Y. Zhang, C. Jiang, and Y. Yang, "3dgs2: Near second-order converging 3d gaussian splatting," in *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*, 2025, pp. 1–10.

[32] X. Liu, D. Shi, and S. Yang, "Mobilegaussian: Efficient 3d gaussian-based open vocabulary scene understanding on mobile devices," in *Proceedings of the 2025 2nd International Conference on Computer Network and Cloud Computing*, 2025, pp. 76–81.

[33] Q. Picard, S. Chevobbe, M. Darouich, and J.-Y. Didier, "A survey on real-time 3d scene reconstruction with slam methods in embedded systems," *arXiv preprint arXiv:2309.05349*, 2023.

[34] Y. Yao, Z. Luo, S. Li, T. Fang, and L. Quan, "Mvsnet: Depth inference for unstructured multi-view stereo," in *European Conference on Computer Vision (ECCV)*, 2018.

[35] P. Kim, J. Kim, M. Song, Y. Lee, M. Jung, and H.-G. Kim, "A benchmark comparison of four off-the-shelf proprietary visual–inertial odometry systems," *Sensors*, vol. 22, no. 24, p. 9873, 2022.

[36] T. Feigl, A. Porada, S. Steiner, C. Löffler, C. Mutschler, and M. Philippsen, "Localization limitations of arcore, arkit, and hololens in dynamic large-scale industry environments." in *VISIGRAPP (1: GRAPP)*, 2020, pp. 307–318.

[37] R. Mur-Artal and J. D. Tardós, "Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras," *IEEE transactions on robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.

[38] Z. Chen, T. Funkhouser, P. Hedman, and A. Tagliasacchi, "Mobilenerf: Exploiting the polygon rasterization pipeline for efficient neural field rendering on mobile architectures," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 16 569–16 578.

[39] S. Rojas, J. Zarzar, J. C. Pérez, J. Sanchez-Riera, A. Rodríguez, F. Segu, and F. Moreno-Noguer, "Re-rend: Real-time rendering of nerfs across devices," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[40] W. S. Fife and J. K. Archibald, "Improved census transforms for resource-optimized stereo vision," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 23, no. 1, pp. 60–73, 2012.

[41] R. T. Collins, "A space-sweep approach to true multi-image matching," in *Proceedings CVPR IEEE computer society conference on computer vision and pattern recognition*. Ieee, 1996, pp. 358–363.

[42] H. Hirschmuller, "Accurate and efficient stereo processing by semi-global matching and mutual information," in *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)*, vol. 2. IEEE, 2005, pp. 807–814.

[43] Y. Chen, J. Jiang, K. Jiang, X. Tang, Z. Li, X. Liu, and Y. Nie, "Dashgaussian: Optimizing 3d gaussian splatting in 200 seconds," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 11 146–11 155.

[44] S. Ren, T. Wen, Y. Fang, and B. Lu, "Fastgs: Training 3d gaussian splatting in 100 seconds," *arXiv preprint arXiv:2511.04283*, 2025.

[45] B. Mildenhall, P. P. Srinivasan, R. Ortiz-Cayon, N. K. Kalantari, R. Ramamoorthi, R. Ng, and A. Kar, "Local light field fusion: Practical view synthesis with prescriptive sampling guidelines," *ACM Transactions on Graphics (ToG)*, vol. 38, no. 4, pp. 1–14, 2019.

[46] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering." *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.